

Report on METRO Near-Real Time Data Warehouse Implementation

Hammad Sarfraz | 21I-1748 | DS-B

Project Overview

The METRO Data Warehouse (DW) is designed to centralize, process, and analyze transactional and master data from METRO stores across Pakistan. Its primary objective is to enable efficient business intelligence by integrating raw transactional data into a well-structured schema, facilitating advanced analysis through OLAP (Online Analytical Processing) queries.

The project employs the MESHJOIN algorithm in the ETL (Extract, Transform, Load) pipeline for near-real-time integration, ensuring timely processing of transactional data and its enrichment with master data. The final processed data resides in the `sales_fact` table, with supporting dimension tables structured in a Star Schema for optimized analytics.

Schema for the Data Warehouse

The METRO DW follows the **Star Schema** architecture, where the central fact table (`sales_fact`) is linked to supporting dimension tables (`products`, `customers`). This schema simplifies complex queries and supports multidimensional analysis.

1. Staging Tables

Staging tables serve as temporary storage for raw data before transformation and loading into dimension or fact tables.

- **transactions_staging:**
 - Raw transactional data, including order details.
 - Attributes: `order_id`, `order_date`, `product_id`, `customer_id`, `quantity_ordered`.
- **customers_staging:**
 - Raw customer data.
 - Attributes: `customer_id`, `customer_name`, `gender`.
- **products_staging:**
 - Raw product data.
 - Attributes: `product_id`, `product_name`, `product_price`, `supplier_id`, `supplier_name`, `store_id`, `store_name`.

2. Dimension Tables

Dimension tables provide clean, deduplicated master data to enrich transactional data.

- **products Table:**
 - Key Attributes: `product_id`, `product_name`, `product_price`, `supplier_id`, `supplier_name`, `store_id`, `store_name`.
 - **customers Table:**
 - Key Attributes: `customer_id`, `customer_name`, `gender`.
-

3. Fact Table

The fact table, `sales_fact`, stores enriched transactional data for analytics.

Attribute	Description
• <code>order_id</code>	Unique Identifier for each transaction.
• <code>order_date</code>	Date of the transaction
• <code>product_id</code>	Foreign Key linking to <code>products.products_id</code>
• <code>customer_id</code>	Foreign Key linking to <code>customers.customers_id</code>
• <code>quantity</code>	No. of items purchased in the transaction.
• <code>product_price</code>	Price per unit of the product.
• <code>total_sale</code>	Derived as <code>quantity * product_price</code>

4. Derived Views

Several views are created for OLAP queries to support high-level analytics:

- **STORE_QUARTERLY_SALES:**
 - Aggregates sales by store and quarter.
 - **YEARLY_REVENUE_ROLLUP:**
 - Aggregates yearly revenue across stores and products.
 - **PRODUCT_MONTHLY_PERFORMANCE:**
 - Evaluates product performance at a monthly level.
-

MESHJOIN Algorithm

The MESHJOIN algorithm powers the ETL pipeline by integrating transactional data with master data efficiently, even for large datasets. It processes data in chunks (partitions) and ensures memory efficiency.

The **MESHJOIN algorithm** is a memory-efficient and scalable approach to integrate large transactional data streams with master data in near-real-time ETL pipelines. It is designed for scenarios where neither the master data nor the transactional data can entirely fit into memory, making it an ideal solution for large-scale data warehouses like METRO DW.

Key Workflow Steps

1. **Disk Buffer:**
 - Master data (e.g., **products** or **customers**) is divided into fixed-size partitions.
 - At any given time, only one partition of master data is loaded into memory (disk buffer) for processing, reducing memory requirements.
2. **Stream Buffer:**
 - Transactions from the incoming stream (e.g., **transactions_staging**) are loaded in small chunks into a **Hash Table** for fast lookups and temporarily stored in a **Queue** to maintain their order.
3. **Join Operation:**
 - Each chunk of transactional data in the stream buffer is joined with the currently loaded master data partition in the disk buffer.
 - Matching transactions are enriched with master data details, and derived attributes, such as **total_sale = quantity * product_price**, are calculated.
 - These enriched records are then inserted into the **sales_fact** table.

4. **Staggered Processing:**

- The **Queue** ensures that transactional chunks remain in memory until they have been joined with all partitions of the master data.
- For example, a transaction in the stream buffer will first join with **Partition 1** of the master data, then **Partition 2**, and so on, until it has matched with the entire dataset.

5. **Partition Swap:**

- Once all transactions in the current stream buffer are processed with the active master data partition, the partition is replaced by the next one in sequence (e.g., **Partition 1** is replaced by **Partition 2**).
- This process repeats cyclically until all master data partitions have been processed with the streamed transactions.
-

Shortcomings of MESHJOIN

1. **Fixed Partition Sizes:**

- Requires predefined partition sizes, which may not adapt well to dynamic workloads or datasets with varying transaction volumes.

2. **Processing Latency:**

- Cyclic processing introduces slight delays, making it unsuitable for scenarios requiring instantaneous joins.

3. **Scalability Limitations:**

- The algorithm struggles to scale when transactional data volumes exceed memory capacity, potentially slowing down processing.

A few OLAP Queries

OLAP queries leverage the DW's Star Schema and MESHJOIN-enriched data to provide actionable insights. Below are key examples:

1. Quarterly Sales by Store

Query:

```
SELECT p.store_name,  
       SUM(sf.total_sale) AS total_sales,  
       QUARTER(sf.order_date) AS quarter
```

```
FROM sales_fact sf

JOIN products p ON sf.product_id = p.product_id

GROUP BY p.store_name, QUARTER(sf.order_date);
```

Insight:

- Tracks store-level performance across quarters.
-

2. Top Products by Revenue

Query:

```
SELECT p.product_name,
SUM(sf.total_sale) AS total_revenue
FROM sales_fact sf
JOIN products p ON sf.product_id = p.product_id
GROUP BY p.product_name
ORDER BY total_revenue DESC
LIMIT 10;
```

Insight:

- Identifies top-performing products, aiding inventory management and promotional strategies.
-

3. Supplier Contribution

Query:

```
SELECT p.supplier_name,
```

```
SUM(sf.total_sale) AS total_sales

FROM sales_fact sf

JOIN products p ON sf.product_id = p.product_id

GROUP BY p.supplier_name;
```

Insight:

- Highlights supplier performance, enabling better negotiations and inventory planning.
-

Key Findings

1. **Revenue Trends:**
 - Cameras and TVs are top-selling products, reflecting seasonal promotions.
 2. **Store Performance:**
 - Significant variability in store-level sales, with some stores outperforming others.
 3. **Supplier Insights:**
 - Canon and LG emerge as top suppliers contributing to significant revenue.
-

What I Learned From This Project

1. **Schema Design:**
 - The importance of a well-structured Star Schema for fast and efficient queries.
 2. **ETL Optimization:**
 - How MESHJOIN balances memory usage and performance for large-scale data processing.
 3. **OLAP Queries:**
 - Crafting queries to extract actionable insights from structured data.
 4. **Data Integration Challenges:**
 - Handling deduplication, missing values, and large datasets efficiently.
-

Conclusion

The METRO Data Warehouse demonstrates the practical application of data warehousing techniques to solve real-world analytical problems. It efficiently integrates raw transactional data with master data using MESHJOIN, providing a robust Star Schema for OLAP queries.

While the MESHJOIN algorithm has limitations (e.g., fixed partition sizes and latency), it effectively supports near-real-time processing. Future enhancements could include dynamic partitioning, real-time updates, and predictive analytics to further improve the system's capabilities.

This project has been an excellent learning opportunity in database design, ETL processes, and business intelligence systems. Let me know if you need additional details or elaboration!